

Học Sâu (Deep Learning)

Chương 4: Bắt đầu với Mạng Nơ-ron (Phân loại và Hồi quy)

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Các thuật ngữ cốt lõi (Glossary)
2. Phân loại Nhị phân (IMDb Dataset)
3. Phân loại Đa lớp (Reuters Dataset)
4. Hồi quy Vô hướng (Boston Housing)
5. Tổng kết

Trong phân loại và hồi quy, các kỹ sư Học máy sử dụng thuật ngữ rất nghiêm ngặt:

- **Sample (Mẫu / Đầu vào):** Một điểm dữ liệu đi vào mô hình. Ví dụ: Một bức ảnh, một đoạn văn bản.
- **Target (Mục tiêu / Sự thật):** Kết quả kỳ vọng. Mô hình lý tưởng sẽ dự đoán ra kết quả này theo một tập dữ liệu chuẩn.
- **Prediction (Dự đoán / Đầu ra):** Giá trị đầu ra thực tế từ mô hình của bạn.
- **Loss value (Giá trị mất mát):** Khoảng cách đo lường sự chênh lệch giữa Prediction (Dự đoán) và Target (Mục tiêu).

Thuật ngữ về Nhãn (Labels & Classes)

- **Classes (Lớp):** Tập hợp các danh mục khả dĩ trong một bài toán phân loại. (Ví dụ: Chó, Mèo, Chim).
- **Label (Nhãn):** Một phiên bản cụ thể của Class được gán cho một Sample. Ví dụ: Ảnh #1234 có nhãn là "Chó".
- **Ground-truth (Sự thật gốc):** Toàn bộ các targets của tập dữ liệu, thường được con người thu thập và gán nhãn thủ công.
- **Batch (Mini-batch - Lô):** Một tập hợp nhỏ các samples (thường 8-128) được mô hình xử lý cùng lúc để tính gradient. Kích thước batch thường là lũy thừa của 2 để tối ưu bộ nhớ GPU.

Phân loại các bài toán Học máy cơ bản

Có 3 bài toán phổ biến nhất trên dữ liệu dạng vector (bảng):

- ➊ **Phân loại Nhị phân (Binary Classification):** Phân loại đầu vào thành 2 nhóm độc quyền (Ví dụ: Chó hoặc Mèo, Positive hoặc Negative).
- ➋ **Phân loại Đa lớp (Multiclass Classification):** Phân loại đầu vào thành nhiều hơn 2 nhóm. Nếu mỗi mẫu chỉ thuộc 1 danh mục duy nhất, ta gọi là bài toán Single-label Multiclass. (Ví dụ: Nhận dạng chữ số 0-9).
- ➌ **Hồi quy Vô hướng (Scalar Regression):** Dự đoán một giá trị mục tiêu liên tục (vô hướng). (Ví dụ: Dự đoán giá nhà, nhiệt độ).

Chuẩn bị dữ liệu (Data Preprocessing)

Mạng nơ-ron không nhận dữ liệu thô (chuỗi văn bản có độ dài khác nhau). Chúng ta phải biến danh sách từ vựng thành tensor:

- **Mã hóa đa điểm (Multi-hot Encoding):** Biến danh sách thành các vector 0 và 1 (chỉ số xuất hiện). Ví dụ: [8, 5] thành vector toàn 0, ngoại trừ chỉ số 5 và 8 là 1.

```
1 import numpy as np
2 def multi_hot_encode(sequences, num_classes):
3     results = np.zeros((len(sequences), num_classes))
4     for i, sequence in enumerate(sequences):
5         results[i, sequence] = 1.0 # Dat 1 tai cac vi tri tu xuat hien
6     return results
```

Bài toán Đánh giá Cảm xúc (IMDb Dataset)

- **Mục tiêu:** Đánh giá xem một bài bình luận phim (movie review) là Tích cực (Positive - 1) hay Tiêu cực (Negative - 0).
- **Dữ liệu:** 50.000 bài đánh giá từ Internet Movie Database, chia đều 25.000 train và 25.000 test. Cân bằng 50% Positive - 50% Negative.
- **Tiền xử lý dữ liệu:** Từ ngữ được mã hóa thành các số nguyên (integer index). Ta sẽ dùng `num_words=10000` để chỉ giữ 10.000 từ xuất hiện phổ biến nhất, loại bỏ các từ quá hiếm.

Tải và Khám phá tập dữ liệu IMDb

```
1 from keras.datasets import imdb
2
3 # Tải 10000 từ phổ biến nhất
4 (train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000)
5
6 # Kiểm tra dòng dữ liệu đầu tiên
7 print(train_data[0])
8 # [1, 14, 22, 16, 43, 530, 973, 1622, ... 178, 32]
9 print(train_labels[0])
10 # 1 (Tích cực)
```

Mỗi bài đánh giá đã được Keras mã hóa sẵn thành chuỗi số nguyên. Ta không phải xử lý văn bản thô (raw text) từ đầu.

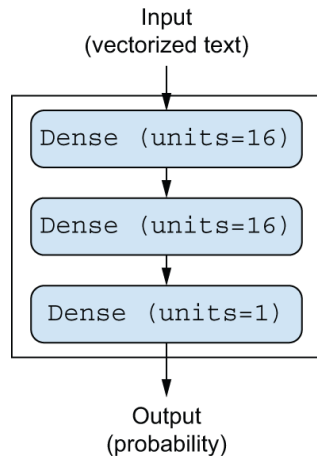
Chuẩn bị Dữ liệu (Vector hóa)

Như đã thảo luận, ta dùng Multi-hot Encoding để biến chuỗi có độ dài thay đổi thành Vector 10000 chiều:

```
1 import numpy as np
2
3 # Vector hóa input (sử dụng hàm multi_hot_encode)
4 x_train = multi_hot_encode(train_data, num_classes=10000)
5 x_test = multi_hot_encode(test_data, num_classes=10000)
6
7 # Label đã là số nguyên 0 hoặc 1, ta chỉ cần chuyển sang float32
8 y_train = train_labels.astype("float32")
9 y_test = test_labels.astype("float32")
```

Kiến trúc Mạng Nơ-ron 3 Lớp

- Mạng có 2 lớp ẩn (Hidden Layers), mỗi lớp có 16 nơ-ron (units).
- Lớp đầu ra (Output Layer) có 1 nơ-ron để đưa ra dự đoán vô hướng (xác suất).
- Ý nghĩa: Lớp thứ nhất trích xuất đặc trưng cơ bản. Lớp thứ hai phân tích quan hệ phức tạp. Lớp cuối tổng hợp kết quả.



Hình 4.1: Cấu trúc mạng 3 lớp cho phân loại IMDb

Định nghĩa Mô hình trong Keras

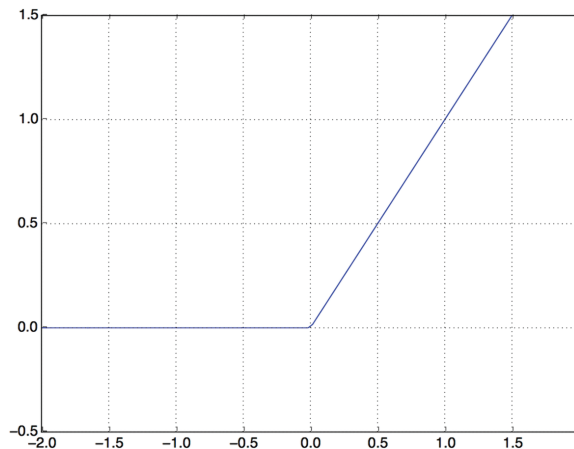
Ta sử dụng `keras.Sequential` để chồng các lớp liên tiếp nhau. Lớp Dense 16 units cần một hàm kích hoạt phi tuyến tính.

```
1 import keras
2 from keras import layers
3
4 model = keras.Sequential([
5     layers.Dense(16, activation="relu"),
6     layers.Dense(16, activation="relu"),
7     layers.Dense(1, activation="sigmoid")
8 ])
```

Hàm kích hoạt (Activation): ReLU

$$f(x) = \max(0, x) \quad (1)$$

- **ReLU (Rectified Linear Unit):** Loại bỏ giá trị âm (trả về 0), giữ nguyên giá trị dương.
- Nếu không có hàm kích hoạt phi tuyến, mạng nhiều lớp cũng chỉ tương đương 1 lớp tuyến tính, không thể giải quyết bài toán phức tạp.

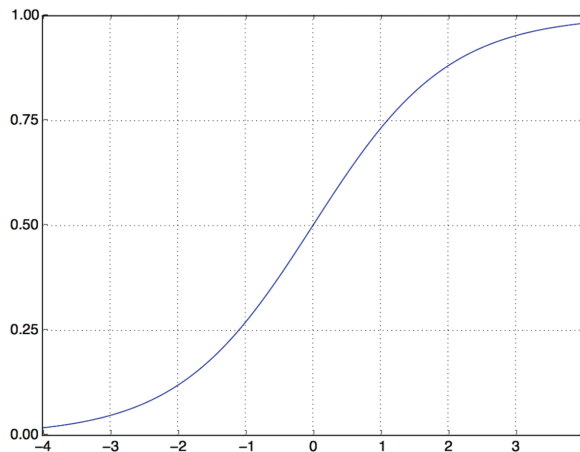


Hình 4.2: Đồ thị của hàm ReLU

Lớp Output: Hàm Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

- Đầu ra của mạng có thể là số bất kỳ $(-\infty, +\infty)$.
- **Sigmoid:** Ép bất kỳ giá trị nào về miền xác suất $[0, 1]$.
- Ví dụ: Kết quả 0.9 $\rightarrow$ Mô hình có 90% tự tin đây là bình luận Tích cực.



Hình 4.3: Đồ thị của hàm Sigmoid

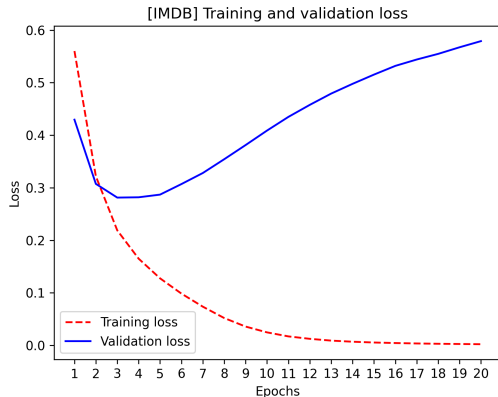
Biên dịch (Compile) và Xác thực (Validation)

Bài toán phân loại nhị phân sử dụng độ đo Loss là `binary_crossentropy`. Thuật toán tối ưu phổ quát là `adam`.

```
1 model.compile(optimizer="adam",
2               loss="binary_crossentropy",
3               metrics=["accuracy"])
4
5 # Huan luyen va theo doi tren tap Validation 20%
6 history = model.fit(x_train,
7                     y_train,
8                     epochs=20,
9                     batch_size=512,
10                    validation_split=0.2)
```

Đánh giá Huấn luyện: Biểu đồ Loss

- Đường chấm bi (Training Loss) liên tục giảm → Mô hình đang học rất tốt dữ liệu đào tạo.
- Đường liền (Validation Loss) giảm xuống mức tối thiểu tại epoch 4, sau đó đột ngột tăng lại.
- **Overfitting (Quá khớp):** Mô hình học thuộc lòng dữ liệu train nhưng không có khả năng tổng quát hóa dữ liệu chưa từng thấy.



Hình 4.4: Sự thay đổi Loss qua các Epochs